

Open Agency “ Technical Specification

Version: 1.0.0

Stack: Node.js • TypeScript • Express • Socket.IO • LangChain • ChromaDB

Runtime: Backend on port 3001 (configurable via PORT env var)

1. System Overview

Open Agency (codenamed **Atlas**) is an autonomous AI agent orchestration platform. It runs a fleet of specialised agents that cooperate through a shared memory system, an event bus, and a task queue – all coordinated by a central orchestrator.

[illegible]

1.1 Entry Point (src/index.ts)

1. Creates an `http.Server` wrapping an Express app with Socket.IO.
2. Initialises `MemorySystem`.
3. Creates `AgentRegistry` and initialises all agents.
4. Creates `AgentOrchestrator`, `TaskRouter`, `EventDispatcher`, `AutonomousScheduler`.
5. Instantiates all integration gateways (Slack, GitHub, Linear, Sentry).
6. Initialises autonomous loops: `ReflectionLoop`, `SelfHealingSystem`, `ResourceAllocator`, `RevenueOptimizationAgent`.

7. Calls `startup()` which calls `.connect()/.start()` on all systems in parallel.
8. Server listens on `PORT`.

1.2 Configuration (`src/config.ts`)

All settings are read from environment variables (dotenv). Defaults:

```
database: { host, port, database, user, password }           // PostgreSQL
llm:      { provider, model, apiKey }
server:   { port: 3001, nodeEnv: 'development' }
frontend: { url: 'http://localhost:3000' }
integrations: { neo4j, gmail, hubspot, notion }
```

2. Agent System

2.1 Base Agent (`src/agents/BaseAgent.ts`)

Every agent extends `BaseAgent`. Core interface:

```
abstract class BaseAgent {
  id: string; name: string; type: AgentType; status: AgentStatus;
  abstract execute(task: Task): Promise<TaskResult>;
  abstract canHandle(task: Task): boolean;
  abstract getCapabilities(): string[];
  abstract getConfig(): AgentConfig;
  setStatus(s: AgentStatus): void;
  initialize(): Promise<void>;
  onTaskComplete(task: Task, result: TaskResult): Promise<void>;
  onTaskError(task: Task, error: Error): Promise<void>;
}
```

`AgentConfig` defaults per agent:

```
{ maxConcurrentTasks: 1€"3, timeoutMs: 30000€"120000, retryAttempts: 1€"3,
  autoApprove: boolean, approvalThreshold: 0€"1000 }
```

2.2 Agent Registry (`src/agents/AgentRegistry.ts`)

- `agents`: `Map<string, BaseAgent>` €" live instances
- `metadata`: `Map<string, Agent>` €" lightweight descriptor objects
- `initialize(memory)`: instantiates all 13 default agents and calls their `.initialize()`

- Key methods: `get(id)`, `getAll()`, `getByType(type)`, `getAvailableAgents()`, `updateMetrics()`, `updateStatus()`, `getStats()`

2.3 Default Agents

Agent	Type	Key Actions	Auto-Approve
CEOAgent	orchestrator	<code>set_strategy,</code> <code>set_goal,</code> <code>analyze_performance,</code> <code>prioritize,</code> <code>allocate_resources,</code> <code>plan_quarter,</code> <code>review_operations</code>	No
MarketingAgent	marketing	<code>create-content,</code> <code>generate-campaign,</code> <code>analyze-seo,</code> <code>generate-hashtags,</code> <code>write-ad, create-report</code>	No
SalesAgent	sales	<code>generate-leads,</code> <code>score-leads, send-outreach,</code> <code>create-sequence, update-pipeline,</code> <code>generate-proposal, analyze-pipeline</code>	No
OperationsAgent	operations	<code>optimize, allocate,</code> <code>generate-report,</code> <code>manage-vendors</code> <code>track-expense,</code> <code>generate-report,</code> <code>analyze-budget,</code> <code>track-revenue,</code> <code>optimize</code>	No
FinanceAgent	finance	<code>web-search, analyze-data,</code> <code>generate-report, track-trend</code> <code>generate-design-system,</code> <code>generate-component,</code> <code>generate-page, apply-brand,</code>	No
ResearchAgent	research		No
DesignAgent	design		Yes

Agent	Type	Key Actions	Auto-Approve
EvolutionAgent	orchestrator	create-icon-set, generate-variations, audit-consistency run-experiment, optimize-workflow, mutate-prompt, analyze-patterns, apply-learning, compare-strategies, generate-insights, evolve-threshold	No
RevenueOptimizationAgent	finance	manage-tiers, run-pricing-experiment, generate-recommendations, analyze-competitors, get-metrics	No
CostMonitorAgent	operations	track, getMetrics, setBudget, checkBudget, suggestOptimizations, generateReport	Yes
TriageAgent	operations	triage, configure, history, set-threshold “ subscribes to EventBus	Yes
PersonalAssistantAgent	personal	schedule, send-email, take-note, remind, search, summarize, draft	Yes
SelfHealingAgent	operations	(part of SelfHealingSystem)	“

2.4 Task Types and Routing

```
type TaskType = 'research' | 'content' | 'outreach' | 'analysis'
               | 'automation' | 'approval' | 'learning' | 'reporting'
```

TaskRouter.routeToBest(task) scores available agents by: - Type match (TASK_TYPE_MAPPING): **+10** - idle status: **+5** - Success rate >90%: **+3** - Expected duration fit: **+2**

3. Memory System (src/memory/MemorySystem.ts)

Two-tier storage:

Tier	Criterion	Storage
Short-term	importance ≥ 0.7 and type = preference	Map<string, MemoryEntry>
Long-term	importance > 0.7 or type = preference	Map<string, MemoryEntry>

Memory entry schema:

```
interface MemoryEntry {
  id: string; type: MemoryType; content: string; embedding?: number[];
  metadata: MemoryMetadata; createdAt: Date; accessedAt: Date;
  importance: number; // 0.0â€“1.0
  tags: string[];
}
type MemoryType = 'interaction' | 'knowledge' | 'preference' | 'context' |
```

Key methods: - store(entry) â†’ auto-selects short/long tier, returns MemoryEntry - search(query, limit) â†’ keyword + tag scoring - consolidate() â†’ promotes frequently-accessed short-term entries after 7 days; demotes long-term entries not accessed in 30 days - getRecent(type?, limit) â†’ reverse-chronological slice

4. Reflection Loop (src/memory/ReflectionLoop.ts)

Autonomous self-improvement cycle running every **1 hour** (configurable):

1. **Collect** last 100 interaction memories and last 50 of any type.
2. **Analyse** success rate and average task duration from recent task memories.
3. **Generate** insights (most frequent task type, failure patterns, memory diversity).
4. **Determine actions** â€” if success < 70%: adjust_threshold (high priority); if avg duration > 5s: refine_workflow (medium); if success > 90%: create_skill (low).
5. **Execute** high-priority actions immediately; defer others.
6. **Store** a knowledge memory with the reflection summary.

Skills created by the loop are stored as type: 'skill' memories and can be retrieved via `getSkills()` / `useSkill(id, context)`.

5. Task Orchestration

5.1 AgentOrchestrator (src/orchestrator/AgentOrchestrator.ts)

- Maintains `taskQueue: Task[]` and `pendingTasks: Map<string, TaskQueueItem>`.
- `start()` begins a **1-second poll interval** on `processQueue()`.
- `createTask()` adds to queue, auto-sorts by priority then creation time, stores a memory entry.
- `requiresApproval()` returns true when `input.spending > 500` (USD).
- `processQueue()` finds the first pending task whose dependencies are all completed, routes it via `TaskRouter`, and calls `executeTask()`.
- `executeTask()` sets agent to busy, calls `agent.execute(task)`, stores result, updates metrics, calls `onTaskComplete` or `onTaskError`, resets agent to active.

5.2 TaskRouter (src/orchestrator/TaskRouter.ts)

Static mapping `TASK_TYPE_MAPPING` aligns `TaskType` â†’ preferred `AgentType[]`.

5.3 TaskGraph (src/orchestration/taskGraph.ts)

DAG structure for dependency management: - `addTask(task)`, `addDependency(taskId, dependsOnId)` - `getReadyTasks(runningIds)` â€” returns executable tasks whose deps are satisfied, sorted by priority - `getExecutionOrder()` â€” topological sort into execution levels (for parallel execution) - `isComplete()`, `hasFailures()`

5.4 AutonomousScheduler (src/events/AutonomousScheduler.ts)

Event-driven task scheduling. Calls `schedule()` to register a future task, `cancel()`, `getScheduled()`. Used by the scheduler to enqueue tasks whose `scheduledAt` time has arrived.

6. Event Bus (src/events/EventBus.ts)

Node.js EventEmitter-based pub/sub. Core type:

```
interface HermesEvent {
  id: string; type: EventType; source: string;
  payload: Record<string, unknown>;
  priority: EventPriority;           // 'low' | 'medium' | 'high' | 'critical'
  timestamp: Date;
  metadata: EventMetadata;          // correlationId, traceId, tags, userId,
}
type EventType =
  'lead_detected' | 'email_received' | 'price_change' | 'metric_threshold'
  | 'task_completed' | 'competitor_detected' | 'conversion_drop'
  | 'revenue_change' | 'system_error' | 'scheduled_trigger'
  | 'slack_command' | 'api_webhook'
```

Subscription modes: - subscribe(eventType, handler) â€” specific event
type - subscribeToAll(handler) â€” all events ('*')

EventDispatcher (src/events/EventDispatcher.ts) dispatches incoming events to appropriate agents based on event type.

7. Core Services

7.1 LLMService (src/services/LLMService.ts)

Wrapper around OpenAI / Anthropic / Grok. Interface:

```
interface LLMResponse { content: string; raw?: unknown; }
complete(prompt: string, opts?: { system?: string }): Promise<LLMResponse>
completeWithContext(prompt: string, context: string): Promise<LLMResponse>
```

Uses the provider and API key from config.llm.

7.2 SelfHealingSystem (src/services/SelfHealingSystem.ts)

- **Health checks every 30 seconds:** agent health, memory health, system resources, task queue.
- **Backup every 1 hour:** full, memory-only, or config-only snapshots stored in a rolling buffer of 24.
- getHealthStatus() returns overall 'healthy' | 'degraded' | 'unhealthy'.

- `attemptRecovery(component)`: resets error-state agents to idle; triggers memory consolidation.
- Backup checksum via simple hash; restore validates checksum before applying.

7.3 ResourceAllocator (src/services/ResourceAllocator.ts)

Tracks CPU and memory utilisation. Key methods: -

`getResourceUtilization()` â†’ `{ cpu: number, memory: number }`
 - `optimizeResourceAllocation()` â†’ returns reallocation recommendations
 - `enableLowCostMode(enabled)` â†’ toggles power-saving profile -
`setResourceCap(cpu, memory)` â†’ hard caps

7.4 MonetizationEngine (src/services/MonetizationEngine.ts)

Tracks revenue streams, clients, campaigns. Key outputs: -

`getFinancialMetrics()` â†’ `{ totalRevenue, totalCosts, netProfit, profitMargin, mrr, arr, churnRate, ltv, cac }` -
`projectRevenue(months)` â†’ MRR projection with 15% monthly growth assumption -
`trackRevenue(streamId, amount)`, `trackCost(streamId, amount)`

8. Autonomous Engines

8.1 AutonomousContentEngine (src/services/AutonomousContentEngine.ts)

- Manages `contentPieces`, `campaigns`, `seoCampaigns`.
- Actions: `create-content`, `schedule-content`, `publish-content`, `analyze-content`, `generate-campaign`, `create-seo-campaign`, `get-analytics`.
- Content pieces have status: `draft` | `scheduled` | `published` | `archived`.
- SEO campaigns track keyword rankings over time.

8.2 AutonomousOutreachEngine (src/services/AutonomousOutreachEngine.ts)

- Manages `leads`, `sequences`, `campaigns`.
- `scrapeLeads(source, criteria)` â†’ generates mock B2B leads using LLM.
- `enrichLead(leadId)` â†’ appends notes and bumps score by up to 10 points.

- `sendOutreach(leadId, sequenceId)` â†’ personalises template variables (`{{name}}`, `{{company}}`, etc.) and sends via email or LinkedIn.
 - Sequences are step-by-step multi-channel (email/LinkedIn/Twitter/cold-call) drip campaigns.
-

9. Integrations

9.1 SlackGateway (`src/integrations/SlackGateway.ts`)

- `connect()` / `disconnect()` â€” mock mode when env vars not set.
- `handleCommand(command)` processes slash commands:

Command	Action
<code>status</code>	System status block
<code>agents</code>	Agent list block
<code>leads</code>	Fetches leads
<code>run <agent> <task></code>	Publishes <code>slack_command</code> event
<code>pause <agent></code>	Pauses agent
<code>approve <task></code>	Approves task
<code>help</code>	Lists commands

- `handleSystemEvent(event)` sends Slack alerts for: `conversion_drop`, `revenue_change`, `competitor_detected`, `system_error`.

9.2 GitHubIntegration (`src/integrations/GitHubIntegration.ts`)

- `createPullRequest({ title, body, head, base })` â†’ returns `GitHubPullRequest` with mock CI checks (CI Test, Lint, Type Check).
- `updatePRStatus()`, `addReview()`, `updateCheck()`, `mergePullRequest()`.
- `getStats()` â†’ open PRs, merged PRs, pending reviews, failed checks.
- Uses `GITHUB_TOKEN`, `GITHUB_OWNER`, `GITHUB_REPO` env vars.

9.3 LinearIntegration (`src/integrations/LinearIntegration.ts`)

- `createIssue({ title, description, priority, projectId, labels })` â†’ returns `LinearIssue`.
- `updateIssue()`, `assignIssue()`, `addComment()`, `closeIssue()`.
- Default projects: `hermes-ops`, `hermes-bugs`, `hermes-features`.
- Priority: `0` = urgent, `1` = high, `2` = medium, `3` = low.

- Uses `LINEAR_API_KEY`, `LINEAR_WORKSPACE` env vars.

9.4 SentryIntegration (src/integrations/SentryIntegration.ts)

- Maintains in-memory issues: `Map<string, SentryIssue>`.
 - `createIssue()`, `resolveIssue()`, `ignoreIssue()`, `getAlertStats()`.
 - `setAlertThreshold(level, count, windowMs)` for custom alert rules.
 - Mock mode by default; real SDK wired via `src/instrument.ts`.
-

10. Tool System (src/tools/ToolRegistry.ts)

Abstract base class `BaseTool` with:

```
abstract class BaseTool {
  id: string; name: string; category: ToolCategory; description: string;
  actions: Map<string, (input) => Promise<ToolResult>>;
  abstract registerActions(): void;
  async execute(action, input): Promise<ToolResult>;
  getToolDefinition(): Tool;
}
```

`ToolRegistry` holds all registered tools. Tools are grouped by category: communication, productivity, data, automation, scraping, analytics.

11. Design System (src/design/)

11.1 CreativeDirectorAgent (src/design/CreativeDirectorAgent.ts)

Orchestrates the design pipeline: brief, generate-concepts, refine-concept, produce-designs, review-designs.

11.2 DesignDNASystem (src/design/DesignDNASystem.ts)

Manages brand design tokens: - **Colors** â€” primary, secondary, success, warning, error, background, surface, text - **Typography** â€” font families, sizes, weights, line-heights - **Spacing** â€” xs through xl (4pxâ€”48px) - **Border radius** â€” sm through xl - **Shadows** â€” sm through xl

11.3 VisualReasoningEngine (src/design/VisualReasoningEngine.ts)

Analyses visual concepts: analyze-image, generate-variations, assess-accessibility, extract-colors.

12. API Routes

All routes are registered on the Express app in `src/index.ts`:

GET/POST /api/agents

- GET / â†’ all agents from registry
- GET /stats â†’ registry stats
- GET /:id â†’ single agent metadata
- POST /:id/start â†’ initialize agent
- POST /:id/stop â†’ set agent to idle

GET/POST /api/tasks

- POST /create â†’ orchestrator.createTask()
- GET / â†’ all tasks (optional ?status= filter)
- GET /:id â†’ single task
- POST /:id/cancel â†’ cancel task

GET/POST /api/memory

- POST /store â†’ memory.store()
- GET /search?q= â†’ memory.search()
- GET /recent?type=&limit= â†’ memory.getRecent()
- POST /consolidate â†’ memory.consolidate()

GET/POST /api/system

- GET /health â†’ self-healing health status
- GET /snapshot â†’ full system snapshot
- POST /backup?type= â†’ create backup
- GET /backups â†’ list backups
- POST /restore/:id â†’ restore from backup
- POST /recover/:component â†’ trigger recovery
- GET /cost/metrics, /cost/budget, /cost/optimize, /cost/report
- POST /cost/track, /cost/budget
- GET /reflection, /reflection/skills

- POST /reflection/trigger
- GET /resources, /resources/optimize
- POST /resources/low-cost, /resources/cap

GET/POST /api/pricing

- GET /tiers â†’ all pricing tiers
- POST /tiers â†’ create tier
- GET /recommendations â†’ price recommendations
- POST /experiments â†’ create pricing experiment
- GET /competitors â†’ competitor analysis

GET/POST /api/monetization

- GET /metrics â†’ financial metrics
- GET /streams â†’ revenue streams
- GET /clients â†’ client list
- POST /clients â†’ add client
- GET /campaigns â†’ ad/content campaigns
- POST /campaigns â†’ create campaign

Socket.IO Events

Event	Direction	Payload
agent:update	serverâ†’client	{ agentId, status, metrics }
task:update	serverâ†’client	{ taskId, status }
event:published	serverâ†’client	HermesEvent
task:submit	clientâ†’server	{ title, description, type, priority, input }
agent:command	clientâ†’server	{ agentId, command, args }

13. Security

13.1 ToolRegistry (src/security/toolRegistry.ts)

Central registry of all tools available to agents. Each tool has an id, name, category, description, and list of actions.

13.2 SecurityPolicyEngine (src/security/policyEngine.ts)

Evaluates security policies for tool execution. Provides: -
evaluateToolAccess(agent, tool) â†’ allow | deny |
require_approval - evaluateDataAccess(agent, resource) â†’ access
level - auditTrail: records all security decisions

13.3 Policies (src/security/policies.ts)

Policy definitions with resource, action, effect (allow/deny), conditions.

13.4 Escalation (src/security/escalation.ts)

Handles security escalations. escalate(event) â†’ creates approval request,
notifies security officer, can auto-revoke if critical.

13.5 Types (src/security/types.ts)

Core types: SecurityPolicy, ToolPermission, DataAccessLevel,
EscalationEvent, AuditEntry.

14. Data Models (Key Interfaces)

Task

```
interface Task {  
  id: string; type: TaskType; priority: TaskPriority;  
  status: TaskStatus; title: string; description: string;  
  assignedTo?: string; dependencies: string[];  
  input: Record<string, unknown>; output?: unknown;  
  result?: TaskResult; createdAt: Date; startedAt?: Date;  
  completedAt?: Date; error?: string; metadata: TaskMetadata;  
}
```

Agent

```
interface Agent {  
  id: string; name: string; type: AgentType; status: AgentStatus;  
  capabilities: string[]; config: AgentConfig;  
  createdAt: Date; lastActiveAt?: Date; metrics: AgentMetrics;  
}
```

UserProfile

```
interface UserProfile {
  id: string; name: string; email: string;
  preferences: UserPreferences;           // communicationStyle, notificati
  businessContext: BusinessContext;       // industry, businessType, curren
  goals: Goal[];                          // id, title, description, target
  createdAt: Date; updatedAt: Date;
}
```

ApprovalRequest

```
interface ApprovalRequest {
  id: string; taskId: string; type: ApprovalType;
  title: string; description: string; details: unknown;
  requestedAt: Date; status: ApprovalStatus;
  resolvedAt?: Date; resolution?: string;
}
type ApprovalType = 'spending' | 'commitment' | 'content' | 'custom'
type ApprovalStatus = 'pending' | 'approved' | 'rejected' | 'expired'
```

15. Environment Variables

Database

DB_HOST=localhost; DB_PORT=5432; DB_NAME=open_agency; DB_USER=postgres; DB

LLM

LLM_PROVIDER=openai # openai | anthropic | grok

LLM_MODEL=gpt-4-turbo-preview

OPENAI_API_KEY=

ANTHROPIC_API_KEY=

Server

PORT=3001

NODE_ENV=development

FRONTEND_URL=http://localhost:3000

Integrations

NEO4J_USER=neo4j; NEO4J_PASSWORD=

GOOGLE_CLIENT_ID=; GOOGLE_CLIENT_SECRET=

HUBSPOT_API_KEY=

NOTION_API_KEY=

GITHUB_TOKEN=; GITHUB_OWNER=; GITHUB_REPO=

LINEAR_API_KEY=; LINEAR_WORKSPACE=

```
# Observability
SENTRY_DSN=https://...@o4511369441574912.ingest.us.sentry.io/4511370553327
```

16. Running the System

```
# Backend
cd src/backend
npm install          # installs @sentry/node, express, socket.io, langchain
npm run dev          # tsx watch src/index.ts (development)
npm run build && npm start  # production

# Frontend (Next.js)
cd src/frontend
npm install
npm run dev

# Docker
docker compose up    # uses docker/Dockerfile and docker-compose.yml
```

17. Sentry Integration

Sentry is initialized in `src/instrument.ts` **before** any other module:

```
import * as Sentry from "@sentry/node";
Sentry.init({
  dsn: process.env.SENTRY_DSN,
  sendDefaultPii: true,
});
```

`src/index.ts` imports `./instrument.js` as its **first line** (before `dotenv/config`), ensuring all unhandled errors and performance traces are captured from startup.

This specification is derived from the source code at `src/backend/src/` and reflects the complete implementation as of commit 2427ac9.